Andrew Pols

(519) 818-1374 | andrew.pols@mail.utoronto.ca | github.com/andrewpols
andrewpols.com | linkedin.com/in/andrewpols

Education

University of Toronto Sep. 2024 - Aug. 2028

- HBSc., **Computer Science and Statistics** | **GPA: 4.00 / 4.00**
- Relevant coursework: Software Design; Computer Organization; Software Tools and Systems Programming

Experience

Software Developer Intern May 2026 - May 2027

IBM Canada Ltd.

- Shipped a dependency graph feature in IBM Planning Analytics Workspace using React and TypeScript, enabling clients to interactively trace relationships in their financial models of over 2,500 objects while staying responsive.
- Reduced a manual trace across thousands of objects from hours to minutes by lazy-loading, caching, and aggregating concurrent asynchronous requests to TM1 REST APIs to parse client scripts and traverse financial object relationships.
- Worked with design, product management, QA, and engineering in Agile sprints to define graph layout, filtering, and object-specific interactions for large dependency structures, then shipped the feature to production for clients.
- Sped up development by 45% by using IBM Bob, an AI coding agent, to run code reviews and generate test cases.

Projects

ScholarSearch | *Source Code* **Python | FastAPI | PostgreSQL | ChromaDB | Redis | ReactJS**

- Engineered a full-stack research paper search engine of over 1,000,000 papers, using PostgreSQL for persistence and ChromaDB for vector embeddings, with Redis caching to serve semantic search queries via FastAPI in under 200ms.
- Constructed D3.js citation graphs so researchers can view a paper's impact and related work in an interactive page.
- Containerized Postgres, ChromaDB, and Redis with Docker so team members shared a consistent development setup.

Multiplayer Pokémon Battle Server | *Source Code* **C | TCP Sockets | UNIX | ncurses**

- Designed a multiplayer Pokémon battle server in C that pairs players into 1v1 matches, serving many concurrent TCP clients from a single-threaded select event loop with a lobby, rematches, forfeits, live chat, and an ncurses CLI UI.
- Defined a binary packet protocol with a 5-byte header consisting of the command byte and length in network byte order, buffering reads so fragmented TCP packets parse correctly and properly notify clients on opponent disconnect.
- Implemented a generic doubly linked list to track clients and active matches, added timestamped connection and match logging, and eliminated memory errors and use-after-free bugs using Valgrind and AddressSanitizer debuggers.

FindMySound | *Source Code* **Python | Django | MySQL | Scikit-Learn | ReactJS**

- Developed a web app that recommends music based on a user's music library, using Spotify's Web API and OAuth 2.0.
- Applied a KNN model on ingested BERT feature vectors to recommend music, reaching 95% surveyed user satisfaction.
- Stored user accounts in a MySQL database through a custom Django user model with JWT-based authentication, and exposed RESTful endpoints for authorization, account syncing, and music recommendations to the React frontend.

GroupFlow | *Source Code* **Java | MongoDB | Swing**

- Created a productivity app with Java Swing and MongoDB where students schedule meetings and assign group tasks.
- Followed Clean Architecture and SOLID design principles, with each user story maintained in its own isolated package.
- Collaborated in a team of 6 developers, enforcing CheckStyle and mandatory code reviews through GitHub actions.

Technical Skills

- **Programming Languages**: Python, Java, C, Go, TypeScript, JavaScript, SQL, HTML5, CSS
- **Frameworks & Libraries**: React, Next.js, FastAPI, Django REST, Flask, Node.js, Scikit-learn, OpenCV
- **Databases**: PostgreSQL, MySQL, MongoDB, Redis, ChromaDB
- **Developer Tools**: Git, Docker, AWS, Azure, UNIX, Valgrind, AddressSanitizer, Nginx, Jira